

α LiDAR: An Adaptive High-Resolution Panoramic LiDAR System

Jiahe Cui[†]

Beihang University, The Chinese
University of Hong Kong
cuijiahe@buaa.edu.cn

Yuze He[†]

The Chinese University of Hong
Kong
yzhh@link.cuhk.edu.hk

Jianwei Niu^{*}

Beihang University
niu Jianwei@buaa.edu.cn

Zhenchao Ouyang

International Innovation Institute
of Beihang University
ouyangkid@buaa.edu.cn

Guoliang Xing^{*}

The Chinese University of Hong
Kong
glxing@cuhk.edu.hk

Abstract

LiDAR technology holds vast potential across various sectors, including robotics, autonomous driving, and urban planning. However, the performance of current LiDAR sensors is hindered by limited field of view (FOV), low resolution, and lack of flexible focusing capability. We introduce α LiDAR, an innovative LiDAR system that employs controllable actuation to provide a panoramic FOV, high resolution, and adaptable scanning focus. The core concept of α LiDAR is to expand the operational freedom of a LiDAR sensor through the incorporation of a controllable, active rotational mechanism. This modification allows the sensor to scan previously inaccessible blind spots and focus on specific areas of interest in an adaptive manner. By modeling uncertainties in LiDAR rotation process and estimating point-wise uncertainty, α LiDAR can correct point cloud distortions resulted from significant rotation. In addition, by optimizing LiDAR's rotation trajectory, α LiDAR can swiftly adapt to dynamic areas of interest. We developed several prototypes of α LiDAR and conducted comprehensive evaluations in

various indoor and outdoor real-world scenarios. Our results demonstrate that α LiDAR achieves centimeter-level pose estimation accuracy, with an average latency of only 37 ms. In two typical LiDAR applications, α LiDAR significantly enhances 3D mapping accuracy, coverage, and density by 8.5 $\times$, 2 $\times$, and 1.6 $\times$ respectively, compared to conventional LiDAR sensors. Additionally, α LiDAR's adaptive rotation improves the effective sensing distance by 1.8 $\times$ and increases the number of perceived objects by 1.9 $\times$. A video demonstration of α LiDAR's in action in real world is available at https://youtu.be/x4zc_I_xTaw. The code is available at https://github.com/HViktorTsoi/alpha_lidar.

CCS Concepts

• Computer systems organization $\rightarrow$ Sensors and actuators; • Computing methodologies $\rightarrow$ Vision for robotics.

Keywords

LiDAR, Active Sensing, Pose Estimation, State Estimation, Sensor Fusion

ACM Reference Format:

Jiahe Cui[†], Yuze He[†], Jianwei Niu^{*}, Zhenchao Ouyang, and Guoliang Xing^{*}. 2024. α LiDAR: An Adaptive High-Resolution Panoramic LiDAR System. In *International Conference On Mobile Computing And Networking (ACM MobiCom '24)*, September 30–October 4, 2024, Washington D.C., DC, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3636534.3690708>

1 INTRODUCTION

LiDAR, which stands for Light Detection and Ranging, has emerged as a critical sensor for efficiently capturing 3D environments, thanks to its accurate range detection, strong interference resistance, and independence from lighting conditions. Its applications in obstacle detection, odometry, and simultaneous localization and mapping (SLAM) are playing an important role in many fields such as robotics [5, 26, 49],

[†] Both authors contributed equally to this work. The order of authorship is interchangeable.

^{*} Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ACM MobiCom '24, September 30–October 4, 2024, Washington D.C., DC, USA © 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0489-5/24/09

<https://doi.org/10.1145/3636534.3690708>

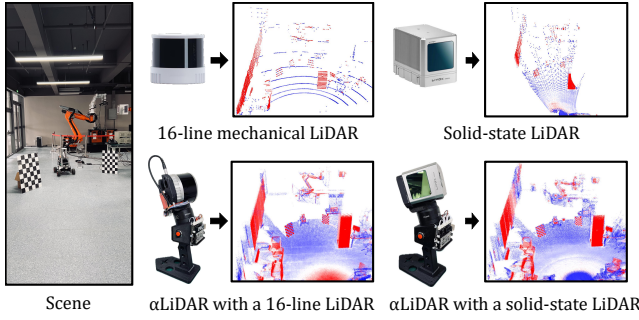


Figure 1: Point clouds generated by α LiDAR systems and native mechanical and solid-state LiDARs.

autonomous driving [19, 34], smart cities [8, 25], and surveying [7, 29, 55]. The global LiDAR market size is expected to grow from USD 1.4 billion in 2023 to USD 3.7 billion by 2029, underscoring its increasing significance [27].

Despite the evident potential and growing market size of LiDAR technology, its widespread adoption across various sectors remains gradual [21]. On one hand, the high cost of LiDAR systems stands as a significant barrier. Especially, high-end LiDARs remain substantially more expensive than other 3D sensors such as binocular camera, depth camera and mmWave radar. On the other hand, even the most advanced LiDARs today still face two major limitations: (i) Limited field of view (FOV) and resolution. Most LiDARs have a narrow FOV, particularly in the vertical direction, where it typically extends only to 40° . Additionally, the limited number of laser beams restricts the resolution of point clouds. These shortcomings create perception blind spots, compromising the system’s ability to thoroughly capture and detail the surrounding environment. (ii) Inflexible focus. In a range of robotic applications, the significance of distinct areas within a scene can shift dramatically. For example, a surveillance robot might initially scan its surrounding environment broadly for navigation. Then, its focus might be narrowed down to areas requiring detailed inspection or suspicious objects. This necessitates LiDARs to perform concentrated, non-repetitive scans of regions of interest (ROIs) to ensure complete and denser coverage of ROIs. However, existing LiDARs are rigidly mounted and follow a fixed scanning pattern, often struggling to adequately cover ROIs and resulting in equally sparse data across the scene. Several solutions are adopted to enhance current LiDAR sensors, such as adding more laser beams [1, 31, 42] or using more LiDAR units [16, 17, 40]. These methods can improve FOV and resolution to some extent, but the cost grows significantly. Moreover, they still fail to offer flexible focus control.

In this paper, we propose enhancing existing LiDAR systems by incorporating the rotation of the LiDAR sensor in the direction where the LiDAR sensor’s field of view is inherently limited. This idea is inspired by human binocular vision, which, despite having an FOV of $120^\circ \times 25^\circ$, enables

complete 360° environmental awareness through the coordinated movement of the eyes and head—an efficient and cost-effective perception mechanism that has developed during human evolution. Mirroring this, LiDAR acts as the “eye” for robots, while α LiDAR functions as the “head,” providing an external active motion structure that expands the LiDAR’s freedom of movement. This design effectively eliminates blind spots and improves the density of point clouds, ensuring extensive scene coverage. Furthermore, it allows for precise control of LiDAR rotation, enabling the system to selectively focus on and thoroughly scan specific ROI for superior quality of perception, akin to the human eye’s ability to fixate. Fig. 1 illustrates that our system enhances the FOV and resolution for both types of LiDAR sensors and enables targeted focused scanning of objects within the scene.

Although such a controllable rotating LiDAR system has great potential to enhance LiDAR performance, its practical implementation is met with several technological challenges. Beyond the hardware design challenges required for controllable rotation and data synchronization, one major issue is that the rapid rotation of α LiDAR is far more intense than the motion typically encountered in standard LiDAR applications. This leads to highly distorted point clouds, with significant translation and rotation differences between consecutive frames, and minimal overlap (less than 50%) between them. In contrast, standard LiDAR applications, like SLAM [36, 38, 54], generally involve only large translations with much higher overlaps (usually over 80%) between frames. Therefore, the key challenge is how to accurately estimate the LiDAR sensor’s pose (state) under rapid rotation to effectively correct and reconstruct the point cloud. However, existing pose estimation methods [12, 47, 48, 50, 52] struggles with α LiDAR. This is because they only model single source of uncertainty, i.e., the sensor data, and assume uniform uncertainty across all points in the point cloud. However, as the LiDAR sensor rotates, the uncertainty of points in different directions of the point cloud varies greatly and changes dynamically.

In this paper, we propose α LiDAR, the first LiDAR system that employs controllable actuation to achieve panoramic FOV, high resolution, and adaptive scanning focus. It innovatively combines software and hardware designs to handle rapid motion dynamics of LiDAR sensors. First, we propose a pose estimation algorithm tailored for α LiDAR that comprehensively models the various sources of uncertainty encountered during LiDAR movement and calculates the uncertainty for each point in the point cloud individually. As a result, it enables accurate and robust state estimation even in the presence of significant motion. Second, to achieve flexible focus control and adaptation to changing ROIs, we propose a rotation planning algorithm customized for actuated LiDAR. This algorithm models the mapping between ROIs and

rotation strategies as an optimization problem. We exploit the fixed rotation axis to narrow down the searching space, allowing for efficient and comprehensive scanning of ROIs through a carefully designed objective function. Third, we design a hardware platform consisting of a LiDAR sensor, motor, IMU, stand, and a control board to enable controllable rotation. The control board, along with its firmware, is carefully designed to ensure real-time synchronization among multiple heterogeneous sensors, and coordinate multi-sensor data transmission. Achieving this synchronization and coordination is particularly challenging when implementing on a low-cost, single-threaded MCU.

α LiDAR offers several advantages that make it an attractive LiDAR enhancement solution: (i) Low cost: α LiDAR only adds about USD \$150 to the inherent cost of the LiDAR itself. (ii) Non-intrusive: α LiDAR does not involve altering the LiDAR sensor itself (e.g., change its internal structure or mode of operation, etc). (iii) Plug and play: α LiDAR is a universal system that's easy to install and remove, compatible with numerous LiDAR models on the market. It features automatic online calibration between the LiDAR and IMU, simplifying setup by removing the need for manual calibration. This allows users to bypass complex and time-consuming calibration process, significantly enhancing α LiDAR's usability and making it more accessible and user-friendly.

We have implemented several prototypes of α LiDAR with different models of LiDAR sensors. We conducted comprehensive evaluation in 7 different real-world scenarios, including indoor, outdoor, and hybrid environment settings, containing a total of 48,677 point cloud frames and covering a LiDAR trajectory of 7.5 km. The evaluation demonstrates that α LiDAR achieves a pose trajectory error of less than 2 cm in indoor environments, and an average error of around 7 cm in all environments. The end-to-end experiments show that when coupled with our system, the FOV of conventional LiDAR sensors can be improved by 8 $\times$, with an average latency of only 37 ms. At the application level, α LiDAR significantly enhances 3D mapping accuracy, map coverage, and density by 8.5 $\times$, 2 $\times$, and 1.6 $\times$, respectively. Moreover, the adaptive rotation feature of α LiDAR extends the effective sensing distance of traditional LiDAR by 1.8 $\times$, resulting in more comprehensive and uniform representation of 3D environments. A video demonstration showcasing α LiDAR in action in real-world scenarios is available at https://youtu.be/x4zc_I_xTaw.

2 RELATED WORK

LiDAR Technology. LiDAR, or Light Detection and Ranging, leverages the Time of Flight principle to measure distances. Current LiDAR models can be broadly categorized into mechanical and solid-state LiDARs.

Mechanical LiDARs use moving parts to direct laser pulses across the environment, typically by rotating mirrors in a manner where the laser beams are vertically aligned and spin to cover a wide horizontal FOV, usually 360°. However, this design often results in a limited vertical FOV and low vertical resolution. As mechanical LiDAR technology has evolved, the number of laser lines has increased from a single line [2, 35] to as many as 128 lines [15, 22, 23, 31, 33, 43] in last two decades, showing the growing demand for higher point cloud resolution. Yet, this improved resolution comes at a significant cost, with the latest 128-line LiDAR [1] exceeding USD \$80,000. Despite the increase in laser lines, the vertical FOV for LiDARs has not seen a corresponding significant improvement, which remains at around 40° even for these advanced models. Solid-state LiDARs [24, 32] offer a more cost-effective solution. They eliminate moving parts by using technologies like optical phased arrays to steer the laser beams, making them more compact, energy-efficient, and capable of achieving high resolution, comparable to up to 180-line of mechanical LiDAR [24]. However, this design inherently not only restricts their ability to perform omnidirectional horizontal scanning but also results in a limited vertical FOV.

Given these constraints, many applications typically use multiple LiDARs to obtain a wider FOV and higher resolution. For example, robotics applications utilize more than one LiDARs for robust 3D mapping [10, 17] and object detection [4, 16], and autonomous driving vehicles often ship with multiple LiDARs to enhance environmental perception and increase driving safety [39, 40, 44].

High-Precision LiDAR Odometry. LiDAR odometry [37, 45, 53, 56] refers to the process of estimating the changes in position and orientation of a LiDAR sensor over time, accomplished by aligning successive point cloud scans or features to determine the sensor's movement. The accuracy of LiDAR-only odometry is satisfactory only when LiDAR undergoes slow motion and observes rich geometry features. However, its performance struggles in high-speed scenarios and feature-poor areas [3, 18]. Achieving more accurate and reliable odometry across a wider range of conditions often requires more than just visual data. Specifically, LiDAR-inertial odometry [12, 38, 47, 48, 50, 52] has been widely adopted in recent years, which merges LiDAR's accurate distance measurements with the high-frequency motion information from IMUs. A common paradigm begins with using IMU data for preliminary motion estimates, then refines the estimations by aligning the current point cloud frame with a local map. It shows robustness when encountering feature-less regions. However, in extreme situations where LiDAR undergoes fast rotation like α LiDAR, noise and bias in IMU data increase due to rapid motion, leading to inaccurate initial motion estimates. Moreover, significant distortions in point cloud

data complicate the matching process, presenting significant challenges for LiDAR-inertial odometry.

3 A MOTIVATIONAL CASE STUDY

This subsection presents a case study to understand the performance of existing LiDAR systems. While our real-world evaluation is presented in Section 7, the case study in this section is conducted in Webots [6], a popular 3D robot simulator that is widely used in industry. We implemented the designs of common LiDAR systems as well as our α LiDAR prototype in Webots.

We evaluated several LiDAR configurations: (a) a 16-line mechanical LiDAR, (b) a 32-line mechanical LiDAR, (c) a solid-state LiDAR, (d) 2 side-by-side solid-state LiDARs, and (e) our α LiDAR with a 16-line mechanical LiDAR sensor. (a) and (c) are representative of two mainstream LiDAR types, whereas (b) and (d) illustrate typical enhancement strategies for increasing the number of internal laser beam or LiDAR units. (e) serves as a usage example of α LiDAR, specifically when integrated with (a). We set up a robot and a typical indoor environment with objects such as tables, chairs, and stairs (Fig. 2(a)). The robot is in front of the stairs and aims to navigate around the stairs from the right, as indicated by the green arrow, making the highlighted red box area its region of interest (ROI). We equip the robot with each LiDAR system to evaluate their performance. The visualizations show point clouds of the entire scene and the ROI, correlating with key robotics tasks—3D mapping and obstacle detection/avoidance. In Table 1, we list their costs and numerical results, with ‘coverage’ and ‘density’ indicating scanning completeness and resolution, respectively.

Limitations of existing solutions. Mechanical LiDARs, both the 16-line and 32-line variants, feature narrow vertical FOV, capture only 37% and 50% of the scene, respectively, resulting in incomplete 3D mapping. In addition, the vertical arrangement of the laser beams leads to linearly shaped, sparsely distributed point clouds that become sparser with distance. This limits the effective identification of obstacles within the ROI, as well as the clarity of mapping. Although the 32-line LiDAR offers an improved resolution by doubling the laser count, it is still too sparse to help the robot detail the ROI and the whole environment. Moreover, this increase in laser beams brings a quadrupling of costs. Solid-state LiDARs, while providing denser point clouds within their FOV for enhanced object perception, also exhibit restricted FOV, covering only 21% of the scene with a single unit. Such inadequacy can pose risks; for instance, as shown in Fig. 2(d), the robot fails to detect obstacles like chairs, potentially resulting in a collision. Doubling the LiDAR units improves the FOV, yet the coverage reaches to only 45%, still significantly limiting 3D mapping efficiency. Compounding the issue, the

Table 1: Specifications of different LiDAR systems.

LiDAR system	FoV	Coverage (ALL)	Density (ALL)	Coverage (ROI)	Density (ROI)	Cost
16-Line	$30^\circ \times 180^\circ$	37.29%	159.91	33.54%	293.37	\$1,150
32-Line	$42^\circ \times 180^\circ$	49.67%	185.70	46.73%	346.62	\$4,500
Solid-state	$72^\circ \times 77^\circ$	21.38%	467.05	26.07%	611.97	\$1,420
Solid-state $\times 2$	$72^\circ \times 154^\circ$	45.45%	440.41	76.48%	526.54	\$2,840
16-Line+ α LiDAR	$300^\circ \times 360^\circ$	72.21%	502.53	89.78%	773.53	\$1,150+\$150

irregular boundary contours of the LiDAR’s FOV can lead to discontinuities at the junctures where the two point clouds converge, as illustrated by the fragmented representation of a chair in Fig. 2(e). Additionally, calibration between the LiDARs presents its own set of challenges. Thus, increasing the number of LiDARs introduces several practical difficulties in real-world applications.

Benefits of controllable actuated LiDAR. Our α LiDAR significantly enhances current LiDAR sensors by adding an external controllable rotating structure. First, through rotation, α LiDAR transitions the scanning from lines (mechanical LiDARs) to surfaces and from segment (solid-state LiDARs) to the whole space, enabling complete and fine-grained global perception. With α LiDAR, the global coverage and density of a 16-line LiDAR are nearly doubled and tripled, respectively. Second, by managing its rotation, α LiDAR can specifically strengthen the scanning of ROIs, aiding the robot in more effectively avoiding obstacles and navigating its surroundings. As a result, the coverage and point density within the ROI are significantly boosted to 2.6 $\times$ that of a standard 16-line LiDAR without α LiDAR. Moreover, α LiDAR achieves the highest improvement in density between global scene and ROI at 54%, a remarkable advancement given its already highest global density among all methods. In addition, α LiDAR’s flexible rotation control minimizes near-range blind areas, unlike other baselines that exhibit extensive nearby blind areas (see the blank space around the robot). Finally, these substantial improvements in both global and local scanning are realized with just an additional cost of about USD \$150, which is equivalent to 1/10 or less the cost of a commercial LiDAR, positioning α LiDAR as an economical solution for LiDAR enhancement.

4 CHALLENGES AND SYSTEM OVERVIEW

Motivated by a wide range of application scenarios, we propose α LiDAR, a novel LiDAR system offering panoramic coverage (both horizontal and vertical), high resolution, and adaptive scanning focus. The core idea behind α LiDAR is two-fold: (i) We add an active rotation mechanism to LiDAR sensors, aiming to cover blind spots, expand the FOV and enhance resolution. (ii) We make the rotation customizable and adjustable by controlling rotation trajectory. This flexibility allows α LiDAR to dynamically focus on specific ROI, facilitating targeted and efficient data collection, and thereby optimizing its performance for various applications.

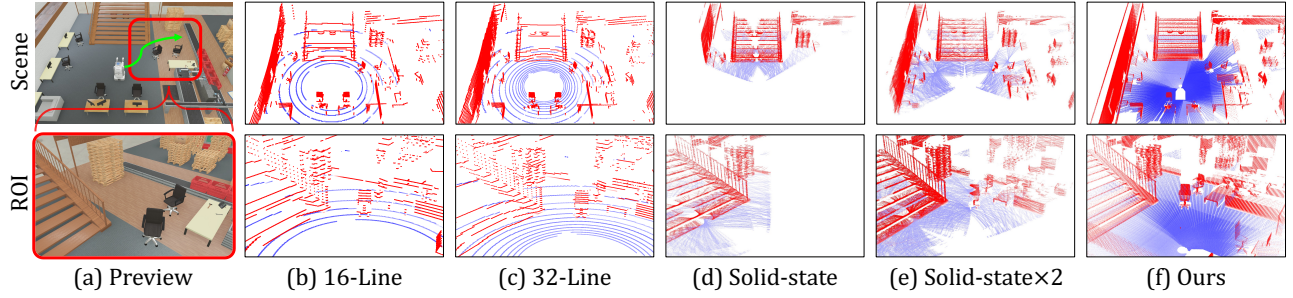


Figure 2: Comparative visualization of full scene and ROI point clouds from various LiDAR systems: Ground points in blue, objects in red.

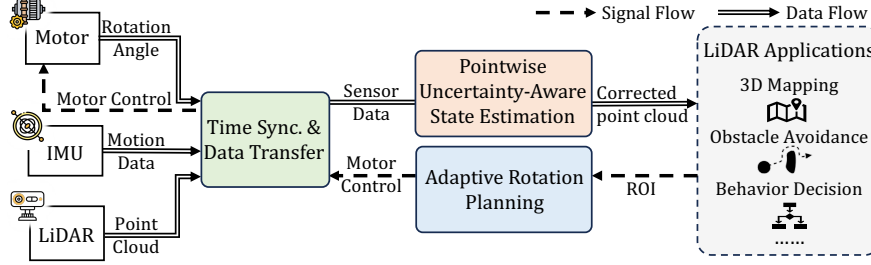


Figure 3: System architecture of α LiDAR.

While α LiDAR exhibits considerable potential for superior performance, its practical implementation faces several challenges. (i) **Accuracy**: In practical applications, the LiDAR typically travels with its carrier (usually a mobile robot or vehicle) while rotating itself (60–300 *rpm*), sequentially capturing 3D points. This movement can lead to significant variations in the coordinate systems of points collected at different times, resulting in distorted point clouds. A common solution is to estimate the LiDAR’s pose (both position and orientation) to realign the points into a uniform coordinate system, thus recovering the captured data. Such pose estimation should be highly accurate, as errors in pose estimation amplify with distance, and even small errors can compromise the position accuracy of points within the point cloud. For example, a 1° angular deviation can lead to a positional error of 1.7 *m* for a point located 100 *m* away. A straightforward approach involves using an IMU to estimate the LiDAR’s pose, but it is limited by the IMU’s accuracy, bias, and drift, leading to unreliable outcomes. LiDAR-inertial odometry [12, 38, 47, 50] offers a more effective solution by aligning the current point cloud with a local map and integrating IMU data for relative pose estimation. This technique works well only when there is significant overlap ($> 80\%$ [28]) between the point cloud and the map, ensuring strong constraints. Nonetheless, the effectiveness of this method faces challenges in our system, where the LiDAR’s rapid movement and rotation reduce this overlap ($< 50\%$). Consequently, developing methods for precise pose estimation, particularly when the LiDAR is subject to significant movements, is a fundamental obstacle. (ii) **Flexibility**: Due to

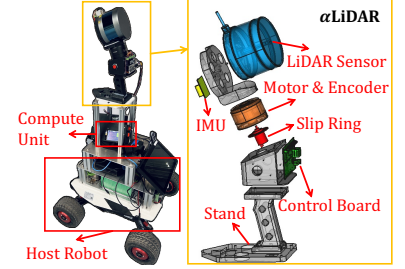


Figure 4: Hardware of α LiDAR.

varying task requirements and environmental conditions, the robot’s ROI is not static but changes dynamically as it navigates. Adapting to the varying ROI necessitates the planning of an optimal LiDAR rotation path to ensure efficient coverage of the targeted sensing area. However, determining this path is challenging due to constantly changing factors, such as the LiDAR’s current rotation angle and the carrier’s future position. Additionally, LiDARs can vary significantly in their FOV. Therefore, planning for rotation must account for these system dynamics and ensure compatibility with various LiDAR FOV models. (iii) **Efficiency**: Unlike conventional LiDAR systems, α LiDAR requires additional computational steps for state estimation, which can delay the delivery of point clouds to downstream applications. Therefore, the state estimation algorithm must be optimized for high efficiency to minimize the latency between data capture and its availability. Moreover, the rotation path planning also needs to be efficient to swiftly adapt to dynamic environments. Achieving such efficiency necessitates novel methods in state estimation and rotation path planning, which collectively underpins α LiDAR’s superior performance.

To tackle the challenges, we first integrate data from multiple sensors—including motor encoder, LiDAR, and IMU—to accurately estimate the LiDAR pose for point cloud calibration. Specifically, we develop a pointwise uncertainty-aware state estimation method that accounts for various sources of uncertainty, including those inherent in LiDAR and IMU measurements, as well as the uncertainty of the extrinsic parameters between the IMU and LiDAR during high-speed rotations. This method meticulously models the uncertainty

for each point in the point cloud, calculates the propagation of these uncertainties through different coordinate systems, and refines the state and uncertainty estimates with each new observation. The result is a state estimation process that is both more robust and accurate, particularly suited for high-dynamic LiDAR systems. Second, achieving highly precise state estimation requires efficient data transfer and real-time synchronization between the sensors. To accomplish this, we develop a hardware bridge component coupled with a synchronization and data transmission mechanism. It is tailored to accommodate the unique synchronization methods and data characteristics (such as size and frequency) of each sensor. Third, we craft an adaptive rotation planning algorithm that determines the optimal rotation path for the LiDAR to concentrate on scanning specific ROIs. This algorithm models the LiDAR’s dynamic scanning process with a set of optimization goals in terms of both scanning completeness and efficiency to identify the best rotation path.

Fig. 3 illustrates the system architecture of α LiDAR. All sensor data, including the motor’s rotation angle data from the motor encoder, acceleration and angular velocity data from the IMU, along with the LiDAR point cloud, are synchronized by the control board and transmitted to the compute unit. Two modules, state estimation and adaptive rotation planning, run concurrently on the compute unit. The state estimation module processes all sensor data to estimate the LiDAR’s pose, undistorts the point clouds, and forwards the corrected point clouds to applications. Concurrently, the adaptive rotation planning module analyzes the specified ROIs from applications and plans the optimal rotation trajectory for the LiDAR. This trajectory is then transmitted via the control board to the motor. We note that α LiDAR is compatible with the interface of current point cloud-based applications. Additionally, α LiDAR provides an API that allows applications or users to specify customized ROIs.

5 DESIGN OF α LiDAR

5.1 α LiDAR Architecture

As depicted in Fig. 4, α LiDAR is composed of two parts: the actuator and the base. Specifically, the actuator includes a LiDAR sensor and an IMU, while the base comprises a stand and a control board. The base is firmly fixed to the carrier, while the actuator is interconnected to the base through a motor. This motor dynamically controls and drives the rotation of the actuator. Inside the motor, we employ a slip ring mechanism to facilitate the transmission of power, data, and control signals to the LiDAR and IMU, avoiding any entanglement issues as the actuator rotates. A motor encoder is attached to the motor to provide information about the position and rotation of a motor shaft. An IMU is mounted

on the LiDAR and moves with it to measure the LiDAR’s motion. A stand supports the LiDAR and secures it to the carrier. We develop a *control board* to synchronize timing across the motor encoder, IMU, and LiDAR, and transmits these data to the compute unit for processing. Additionally, the control board sends control signals from the compute unit to the motor, controlling the rotation of the actuator.

5.2 Time Synchronization and Data Transmission

To achieve precise state estimation, real-time synchronization of data from various sensors is vital. Software-based synchronization methods are typically based on the data receipt time on the host computer, whose accuracy is affected by a variety of factors. For instance, the variance in sensor data volume (e.g., 0.32MB for a LiDAR frame vs. 80B for an IMU frame) and fluctuations in transmission rates inevitably lead to time discrepancies ranging from several to tens of milliseconds. Assuming a LiDAR rotates at a speed of 120 *rpm*, even a 10ms time difference can result in a ranging error of up to 1.3 *m* at a distance of 100 *m*, which is unacceptable for many LiDAR applications. Therefore, it is essential to achieve real-time synchronization (around 100 microseconds) among different sensors. However, achieving these requirements on low-cost embedded platforms presents several challenges. (i) The motor encoder, IMU, and LiDAR are heterogeneous systems, each with its unique timing mechanism; (ii) Embedded devices often face limitations in peripheral resources, computational power, and memory. Such constraints can pose challenges to simultaneously processing large volumes of data from multiple sensors for time synchronization. We address these challenges through a coordinated design of hardware, synchronization mechanisms, and data input/output methods.

We devised a control board serving as a conduit between the sensors and the compute unit, facilitating precise time synchronization among the sensors and centralized data transmission. Given that LiDAR units often possess internal clocks and typically achieve time synchronization through GNSS signals, our control board’s MCU simulates GNSS navigational data, including precise time. This allows the LiDAR to synchronize its internal clock with the MCU’s time encapsulated in the GNSS message, enabling the direct transmission of synchronized point clouds to the compute unit. In contrast, the IMU and motor encoder lack internal clocks; here, we employ interrupts to realize time synchronization. After each periodic angle measurement, the encoder sends angular data to the MCU and triggers a transmission interrupt. The MCU records the system time of this interrupt and store the time-stamped angle data in its cache. When the IMU completes a sampling, it notifies the MCU with a

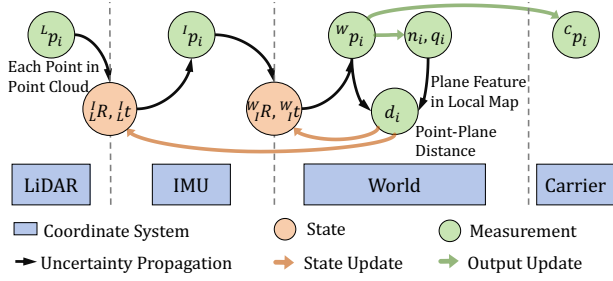


Figure 5: Pointwise uncertainty-aware state estimation. ‘data Ready’ signal. The MCU records the system time and retrieves the IMU data via a serial port. This time-stamped IMU data, combined with cached encoder data, forms a packet and is sent to the compute unit via an upstream serial port. Considering the IMU’s rapid sampling rate and the serial port’s limited speed, a blocking transmission of IMU data could lead to delays exceeding the IMU’s sampling interval, thereby pausing other MCU tasks and disrupting synchronization with other sensors. To circumvent this, we employ DMA to facilitate efficient transmission of IMU data.

5.3 Precise LiDAR Pose Estimation

Upon receiving sensor data, the system must estimate the LiDAR’s precise state (pose) for point cloud de-distortion. However, as introduced in Sec. 4, conventional LiDAR-inertial odometry may fail in our system due to unreliable point cloud registration during rapid LiDAR rotation. To overcome this, it’s critical to incorporate uncertainty into the state estimation framework. While some existing works [12, 47, 48] account for uncertainty in pose estimation, they often assume uniform uncertainty across sensor data. In contrast, our approach carefully evaluates the uncertainty of each point in the point cloud individually, enhancing robustness and accuracy in highly dynamic LiDAR systems. Additionally, given the high-speed rotation of both the IMU and LiDAR in our system, material deformation and initial calibration errors can skew the extrinsic parameters between IMU and LiDAR. We thus propose integrating the uncertainty of IMU and LiDAR extrinsics into our uncertainty estimation framework. Specifically, we employ a Kalman filter to merge LiDAR and IMU data for state estimation, which models uncertainty from various sources and analyzes uncertainty propagation to maintain robustness and precision under rapid rotation. Fig. 5 illustrates the processing pipeline of our method. Below, we detail the uncertainty modeling, the derivation of its propagation, and the update of state and uncertainty.

We define the predicted state $\mathbf{x}_t$ at time t and its uncertainty $\mathbf{U}_t$ as

$$\mathbf{x}_t = [\mathbf{R}_I^W, \mathbf{t}_I^W, \mathbf{R}_L^I, \mathbf{t}_L^I], \mathbf{U}_t = [\Sigma_{\mathbf{R}_I^W}, \Sigma_{\mathbf{t}_I^W}, \Sigma_{\mathbf{R}_L^I}, \Sigma_{\mathbf{t}_L^I}] \quad (1)$$

where $\mathbf{R}_I^W, \mathbf{t}_I^W$ represent the IMU’s pose (including orientation and position) in the world coordinate system, which are predicted by adding the IMU integration from time $t - 1$

to t to the IMU pose at time $t - 1$. $\Sigma_{\mathbf{R}_I^W}, \Sigma_{\mathbf{t}_I^W}$ represent their uncertainty, respectively. $\mathbf{R}_L^I, \mathbf{t}_L^I$ are the extrinsic parameters between the IMU and LiDAR, and $\Sigma_{\mathbf{R}_L^I}, \Sigma_{\mathbf{t}_L^I}$ are their uncertainty, respectively.

According to LiDAR’s measurement model and noise model [51], we can derive the uncertainty $\Sigma_{\mathbf{p}_i}$ of each point $\mathbf{p}_i$ in the point cloud. Based on the IMU-LiDAR extrinsic parameters, point $\mathbf{p}_i$ can be transformed from the LiDAR coordinate system to the IMU coordinate system:

$$\mathbf{p}_i = \mathbf{R}_L^I \mathbf{p}_i + \mathbf{t}_L^I \quad (2)$$

Considering the uncertainty of IMU-LiDAR extrinsic parameters, after the above transformation, the uncertainty of the point in the IMU coordinate system is:

$$\Sigma_{\mathbf{p}_i} = \mathbf{R}_L^I \Sigma_{\mathbf{p}_i} \mathbf{R}_L^{IT} + \mathbf{R}_L^I \left[\mathbf{p}_i \right]_{\wedge} \Sigma_{\mathbf{R}_L^I} \left[\mathbf{p}_i \right]_{\wedge}^T \mathbf{R}_L^{IT} + \Sigma_{\mathbf{t}_L^I} \quad (3)$$

where $[\cdot]_{\wedge}$ is the skew operator.

Furthermore, based on the IMU’s pose in the world coordinate system $\mathbf{R}_I^W, \mathbf{t}_I^W$, and their uncertainty $\Sigma_{\mathbf{R}_I^W}, \Sigma_{\mathbf{t}_I^W}$, we can transform the point $\mathbf{p}_i$ and propagate its uncertainty $\Sigma_{\mathbf{p}_i}$ from the IMU coordinate to the world coordinate system:

$$\mathbf{p}_i = \mathbf{R}_I^W \mathbf{p}_i + \mathbf{t}_I^W \quad (4)$$

$$\Sigma_{\mathbf{p}_i} = \mathbf{R}_I^W \Sigma_{\mathbf{p}_i} \mathbf{R}_I^{WT} + \mathbf{R}_I^W \left[\mathbf{p}_i \right]_{\wedge} \Sigma_{\mathbf{R}_I^W} \left[\mathbf{p}_i \right]_{\wedge}^T \mathbf{R}_I^{WT} + \Sigma_{\mathbf{t}_I^W} \quad (5)$$

In the world coordinate system, we use the plane feature matching error between the current point cloud frame and the local map as constraints to update the state $\mathbf{x}_t$ at time t and its uncertainty $\mathbf{U}_t$. Local maps often include processed features such as plane features, edge features, or other geometric structures extracted from past point cloud data.

For each point $\mathbf{p}_i$ in the point cloud, we search for the nearest planar feature $[\mathbf{n}_i, \mathbf{q}_i]$ in the local map, where $\mathbf{n}_i$ is the normal vector of the plane, and $\mathbf{q}_i$ is the centroid of the planar feature. Therefore, the distance from $\mathbf{p}_i$ to the planar feature is:

$$d_i = \mathbf{n}_i^T (\mathbf{p}_i - \mathbf{q}_i) = \mathbf{n}_i^T (\mathbf{R}_L^I \mathbf{p}_i + \mathbf{t}_L^I - \mathbf{q}_i) \quad (6)$$

The uncertainty of this distance d_i comes from two sources: one is the uncertainty $\Sigma_{\mathbf{p}_i}$ of the point $\mathbf{p}_i$ itself, and the other is the uncertainty of planar features in the local map, $\Sigma_{\mathbf{n}_i}$ and $\Sigma_{\mathbf{q}_i}$. Based on this, we can calculate the observation variance of d_i :

$$\mathbf{Q}_i = \begin{bmatrix} \mathbf{J}_{\mathbf{n}_i} \\ \mathbf{J}_{\mathbf{q}_i} \\ \mathbf{J}_{\mathbf{p}_i} \end{bmatrix}^T \begin{bmatrix} \Sigma_{\mathbf{n}_i} & \mathbf{0} \\ \mathbf{0} & \Sigma_{\mathbf{q}_i} \\ \mathbf{0} & \Sigma_{\mathbf{p}_i} \end{bmatrix} \begin{bmatrix} \mathbf{J}_{\mathbf{n}_i} \\ \mathbf{J}_{\mathbf{q}_i} \\ \mathbf{J}_{\mathbf{p}_i} \end{bmatrix}, \quad (7)$$

$$\mathbf{J}_{\mathbf{n}_i} = \left(\mathbf{R}_L^I \mathbf{p}_i + \mathbf{t}_L^I - \mathbf{q}_i \right)^T, \quad (8)$$

$$\mathbf{J}_{\mathbf{q}_i} = -\mathbf{n}_i^T, \text{ and } \mathbf{J}_{\mathbf{p}_i} = \mathbf{n}_i^T.$$

$\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}$, and $\mathbf{J}_{\mathbf{p}_i}$ are the Jacobian matrices of d_i with respect to $\mathbf{n}_i, \mathbf{q}_i$, and $\mathbf{p}_i$, respectively.

For the whole point cloud frame, we can get the observation function as $h(\mathbf{x}_t) = [d_0, d_1, \dots, d_N]$, and the observation variance as $\mathbf{Q} = [\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_N]$. Ideally, if the predicted state $\mathbf{x}_t$ is accurate, then each point-to-plane distance d_i should be 0. Therefore, we use the IKFoM method [47] to update the state by minimizing the point-to-plane distance. First, we compute the Kalman Gain:

$$\mathbf{K} = (\mathbf{H}^T \mathbf{Q}^{-1} \mathbf{H} + \mathbf{U}_t^{-1})^{-1} \mathbf{H}^T \mathbf{Q}^{-1} \quad (9)$$

where $\mathbf{H}$ is the Jacobian matrix of the observation function $h(\mathbf{x}_t)$ with respect to the state $\mathbf{x}_t$. Then, we update the estimation of state $\mathbf{x}_t^*$ and its uncertainty $\mathbf{U}_t^*$:

$$\mathbf{x}_t^* = \mathbf{x}_t - \mathbf{K}h(\mathbf{x}_t), \text{ and } \mathbf{U}_t^* = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{U}_t. \quad (10)$$

Based on $\mathbf{x}_t^*$ and $\mathbf{U}_t^*$, we recalculate the point ${}^W\mathbf{p}_i^*$ and its uncertainty ${}^W\Sigma_{\mathbf{p}_i}^*$ in the world coordinate system according to formulas (2) to (5). We incorporated the updated ${}^W\mathbf{p}_i^*$ and ${}^W\Sigma_{\mathbf{p}_i}^*$ into the local map to update the corresponding planar features $\mathbf{n}_i^*$ and $\mathbf{q}_i^*$, and their uncertainties $\Sigma_{\mathbf{n}_i}^*$ and $\Sigma_{\mathbf{q}_i}^*$.

Additionally, the updated point cloud needs to be transformed into the carrier (i.e. robot) coordinate system to provide input for downstream applications such as obstacle detection. Since the LiDAR only rotates relative to the carrier, the relative rotational pose is solely determined by the motor encoder's rotation angle. We can uniquely determine the relative extrinsic parameters between the LiDAR and the carrier ${}^L\mathbf{R}_\theta$, ${}^L\mathbf{t}_\theta$, based on the encoder's rotation angle θ at time t . Thus, the transformation required to convert point ${}^W\mathbf{p}_i^*$ in the world coordinate system to the carrier coordinate system is:

$${}^C\mathbf{R} = {}^W\mathbf{R}_L^I \mathbf{R}_C^L, \text{ and } {}^C\mathbf{t} = {}^W\mathbf{R}_L^I \mathbf{R}_C^L \mathbf{t}_\theta + {}^W\mathbf{R}_L^I \mathbf{t} + {}^W\mathbf{t} \quad (11)$$

Therefore, we can obtain the point in the carrier coordinate system ${}^C\mathbf{p}_i = {}^C\mathbf{R}^T {}^W\mathbf{p}_i - {}^C\mathbf{R}^T {}^C\mathbf{t}$.

5.4 Adaptive Rotation Planning

Upon receiving a new ROI from the application, α LiDAR is tasked with planning an optimal rotation trajectory for the LiDAR sensor to cover the ROI completely and effectively. The problem can be defined as follows:

$$\text{Maximize: } O(\text{Rot_Traj}) = \text{Coverage} + \text{Efficiency}. \quad (12)$$

This problem differs from the typical trajectory planning in robotics, which optimizes a sequence of trajectory points with a high degree of freedom. In contrast, the rotation of α LiDAR is restricted to a fixed rotation axis with respect to the carrier, with each scanning orientation uniquely corresponding to a specific motor rotation angle. This constraint simplifies the rotation trajectory to minimum and maximum motor rotation angles, thus reducing the optimization space relative to conventional trajectory planning.

Leveraging this, with the minimum and maximum motor rotation angles denoted as θ_a and θ_b , the term *Coverage* can be described as the overlapping volume between the space covered by all point clouds during the rotation from θ_a and θ_b and the ROI space. Similarly, *Efficiency* can be represented by the average overlapping volume between the space covered by each frame of point cloud during the rotation from θ_a to θ_b and the ROI space. We discretize the variable θ and formulate the objective function as follows:

$$O(\theta_a, \theta_b) = \left| \left(\bigcup_{i=a}^b V_{\theta_i} \right) \cap V_{ROI} \right| + \frac{\sum_{i=a}^b |V_{\theta_i} \cap V_{ROI}|}{b - a + 1} \quad (13)$$

s.t. $0^\circ \leq \theta_a < \theta_b < 360^\circ$.

where V_{ROI} is the space within the ROI, represented by a convex hull constructed from a set of 3D vertices specified by the applications. V_{θ_i} is the view space of the LiDAR corresponding to the motor rotation angle θ_i . The operation $|\cdot|$ computes the volume of a 3D space, while $\cap$ computes the intersection over union (IoU) between two 3D space.

To further derive V_{θ_i} , we proceed by modeling the dynamic scanning process of α LiDAR. Exploiting the rotation constraint, we denote the LiDAR orientation corresponding to the motor rotation angle θ as $\mathbf{R}_\theta$, where $\mathbf{R}$ is a 3×3 rotation matrix. The LiDAR's FOV forms a view frustum, defined as a convex hull, which can be calculated based on the LiDAR's horizontal and vertical angular ranges along with its detection distance. Assume an application updates V_{ROI} at time t_0 , with the motor's current rotation angle denoted as θ_0 , and the carrier's pose is ${}^C\mathbf{T}_0$, moving with a linear velocity $\mathbf{v}_0$ and angular velocity $\mathbf{w}_0$. Specifically, for any rotation angle θ_i , $\theta_a \leq \theta_i \leq \theta_b$, we first calculate the time Δt required for the motor to rotate from θ_0 to θ_i . We model the motion of the carrier over such short time interval Δt as uniform motion. Then, we can predict the carrier's pose ${}^C\mathbf{T}_i$ at time t_i based on the carrier's current pose ${}^C\mathbf{T}_0$, its linear speed $\mathbf{v}_0$ and angular speed $\mathbf{w}_0$. The LiDAR's pose can then be derived from ${}^L\mathbf{T}_i = {}^C\mathbf{T}_i \mathbf{R}_{\theta_i}$, the composition of the carrier's pose ${}^C\mathbf{T}_i$ and the LiDAR's orientation relative to the carrier $\mathbf{R}_{\theta_i}$. Next, by transforming the LiDAR's FOV model with ${}^L\mathbf{T}_i$, we can determine the LiDAR's view space V_{θ_i} at time t_i .

Having derived V_{θ_i} , we have now fully defined the objective function in formula (13). To find the optimal values of (θ_a, θ_b) that maximize the objective function, we employ Powell's algorithm [46] for optimization and solution. Once we obtain the values of θ_a and θ_b , the LiDAR performs repeated scans between these two angles to ensure thorough scanning of the ROI. Before a new ROI is specified, we update the values of θ_a and θ_b at a frequency that is dynamically adjusted based on the carrier's velocity. Whenever a new ROI is received, it triggers a new optimization process to determine the optimal values of θ_a and θ_b .

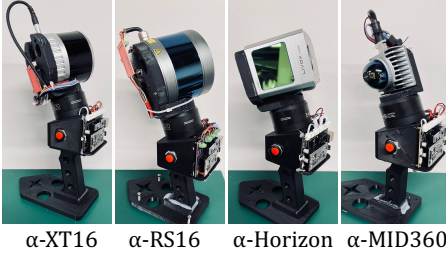


Figure 6: Four prototypes of α LiDAR with different LiDAR sensors.

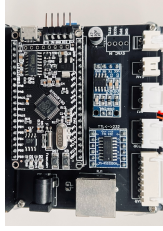


Figure 7: Control board.

6 IMPLEMENTATION & EXPERIMENTAL SETUP

In this section, we present the implementation details of α LiDAR and the experimental setup for its evaluation.

Implementation. We implement α LiDAR with multiple LiDAR sensor models, including Hesai Pandar-XT16, Robosense RS16, Livox Horizon, and Livox MID360, as shown in Fig. 6. We employ DJI GM6020 motors, Yesense YIS100ADK IMUs, control boards, and stands to compose α LiDAR prototypes. Fig. 7 shows our custom-designed control board, featuring an MCU and multiple interfaces, connected sensors, the compute unit, and power supply. The total board cost is only about USD \$10. The time synchronization and data transmission mechanism (Sec. 5.2) is implemented in the firmware flashed onto the MCU.

Experimental Setup. In the evaluation, these prototypes were either mounted on a cart or handheld by a person (to mimic a host robot) at a height of around 1.5 m. The two software modules, state estimation (SE) and adaptive rotation planning (ARP), were executed either on a Raspberry Pi or an Intel NUC11. Here, we leverage these two different computing devices to evaluate the efficiency of α LiDAR.

We conducted a comprehensive evaluation of α LiDAR across a wide range of real-world scenarios, as detailed in Fig. 8, which include indoor, outdoor, and hybrid environments. These environments feature challenging scenes, such as corridors, staircases, indoor-outdoor transitions, which are widely recognized as challenging scenes in state estimation [14]. Altogether, the 7 datasets comprise 48,677 frames of point clouds, covering a total trajectory distance of 7,547 m.

To ascertain the ground truth pose of LiDAR in indoor environments, we use a Leica AT930 laser tracker that provides millimeter-level accuracy to dynamically track the LiDAR system’s trajectory in real time. For outdoor environments, we deploy a high-end UBLOX-F9P RTK-GNSS system [41] with an accuracy of 2 cm for tracking the LiDAR systems’ trajectories. Datasets that include ground truth trajectories are employed for evaluating the accuracy of state estimation, while datasets without ground truth are used for assessing the quality of generated point clouds. Additionally, to assess

α LiDAR’s performance in SLAM applications, we meticulously and repeatedly use LiDAR to manually scan the environment, and then utilize the offline mapping algorithm from [20] to construct the ground truth map.

7 EVALUATION

In this section, we begin by defining the evaluation metrics in Sec. 7.1. Next, we present an end-to-end evaluation of α LiDAR in Sec. 7.2. In Sec. 7.3, we assess the performance of α LiDAR’s key enabling algorithm, the state estimation method. Additionally, we analyze the detailed system overhead of α LiDAR in Sec. 7.4. Lastly, in Sections 7.5 and 7.6, we explore two typical LiDAR applications, 3D mapping and obstacle perception, as case studies to showcase the enhancements these applications can achieve through α LiDAR.

7.1 Evaluation Metrics

7.1.1 ATE, RTE. We use Average Trajectory Error (ATE) and Relative Trajectory Error (RTE) [30] to evaluate the accuracy of the LiDAR trajectory estimated by the pose estimation algorithm. We do not directly evaluate rotation (orientation) estimation errors here for two reasons: 1) Measuring the ground truth orientation of the LiDAR sensor during high-speed rotations is challenging; 2) Errors in orientation estimation significantly impact trajectory accuracy. For instance, a 1-degree angular error can result in a positional error of approximately 0.87 m over a subsequent trajectory of 50 m. Therefore, trajectory accuracy indirectly reflects the precision of orientation estimations.

Specifically, ATE is the discrepancies between the estimated trajectory and the ground truth trajectory: $ATE = \frac{1}{n} \sum_i^n \|t_i^T - t_i^E\|_2$, where n denotes the frame number, and t_i^T and t_i^E are ground-truth LiDAR position and estimated LiDAR position, respectively. For each dataset, we divide the global trajectory into multiple segments of equal length, and the final ATE is the mean of the ATEs for these segments. Besides, we also leverage the Relative Trajectory Error (RTE) from the KITTI Benchmark [9] to evaluate the degree of trajectory drift over time.

7.1.2 Chamfer Distance, Coverage, Efficiency, Density, Uniformity, Feature amount. We adopt these metrics to evaluate the performance of α LiDAR in the SLAM application. The Chamfer Distance (CD) measures the similarity between two point clouds. We compute CD between the SLAM-generated point cloud map X and the ground truth map Y . Thus, a lower CD indicates higher accuracy in map construction. Coverage evaluates the completeness of the SLAM map. It calculates the ratio of covered points N_p to the total number of points N_{gt} in the ground truth map. If the distance between a point p from the ground truth map and its nearest neighbor q in the SLAM map is below

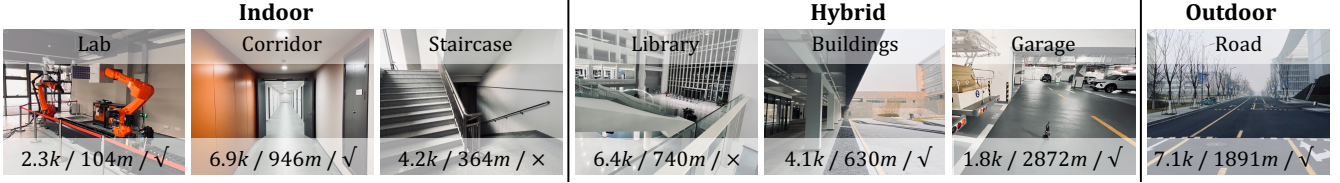


Figure 8: Real-world scenarios for system evaluation. Statistics at the bottom of each scene: Number of point cloud frames / LiDAR trajectory length / Ground truth availability.

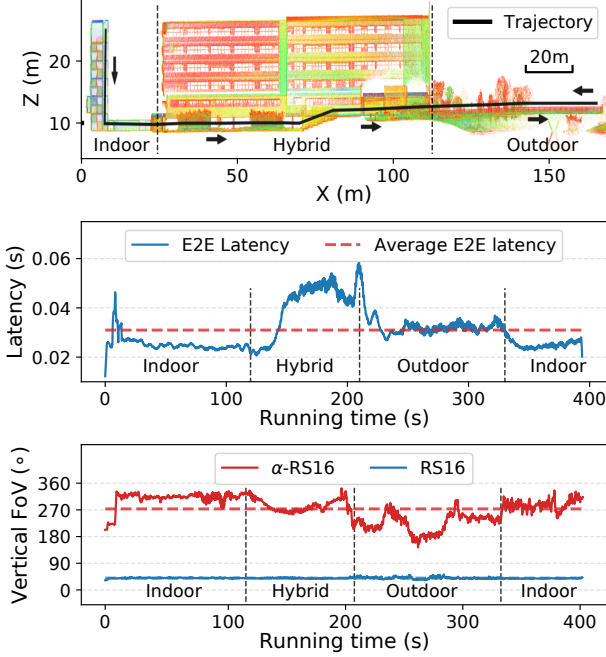


Figure 9: An end-to-end evaluation of α LiDAR. Above: sectional view of the point clouds constructed by α LiDAR. Middle: α LiDAR’s end-to-end latency during the data trace. Bottom: the FoV gain of α LiDAR comparing with conventional LiDAR sensors.

a predefined threshold, p is considered covered by q . It is given by $Coverage = N_p / N_{gt}$, while the uncovered area is $1 - Coverage$. *Efficiency* assesses the proportion of points in a point cloud frame that covers the ground truth map. *Density* represents the average number of points per square meter in the SLAM map, reflecting the map resolution. The uniformity of the SLAM map is assessed through the Cumulative Distribution Function (CDF) of the neighborhood radius of its points. The neighborhood radius of a point q in the SLAM map is defined as the average distance between q and its n nearest neighbors. Additionally, we evaluate the richness of features in the SLAM map by quantifying the quantity of plane feature points [11].

Table 2: State estimation performance comparison between α LiDAR and existing algorithms (ATE in Meters).

Method	Indoor		Hybrid		Outdoor		Average	
	ATE	RTE	ATE	RTE	ATE	RTE	ATE	RTE
LIO-SAM	1.616	18.78%	3.525	10.85%	0.198	2.59%	1.780	10.74%
Point-LIO	0.136	2.59%	0.360	4.66%	0.221	1.69%	0.239	2.98%
α LiDAR	0.018	0.12%	0.022	0.54%	0.167	1.07%	0.069	0.58%

Table 3: Latency of state estimation on different compute platforms.

Platform	Data Trans.	State estimation	Map update	Total
Intel NUC 11	6.2 ms	22.8 ms	7.5 ms	36.5 ms
Raspberry Pi	7.6 ms	64.1 ms	18.2 ms	89.9 ms

7.1.3 ROI density, ESD.. These two metrics are adopted to evaluate the performance of α LiDAR in the obstacle perception application. *ROI density* is the average number of points per cubic meter within the region of interest. We adopt the concept of *ESD* from [13] to differentiate it from the detection range commonly associated with LiDAR systems. *ESD* specifically refers to the range within which the LiDAR device can provide reliable and accurate measurements of objects. This distinction is critical, as *ESD* assesses the usability and quality of the point cloud data.

7.2 End-to-End System Evaluation

As shown in Fig. 9, we collect data traces using α LiDAR (α -RS16, specifically) to evaluate its end-to-end performance. The trace collection starts indoors, moves through stairs and corridors, navigates between multiple tall buildings, reaches outdoor roads, and then returns indoors, as illustrated in the upper figure. A video clip showcasing the real-time operation of α LiDAR is available at https://youtu.be/x4zc_I_xTaw.

The upper figure shows α LiDAR’s point clouds throughout this process, which accurately and completely depicted the details of buildings and trees, reflecting its superior FOV, resolution, and precise LiDAR pose estimation. The lower figure compares the real-time vertical FOV of α -RS16 with RS16, showing an 8× FOV increase for RS16 when coupled with our α LiDAR system. In outdoor environments, the FOV of α -RS16 varies and decreases due to α LiDAR’s adaptive

rotation, which focuses scanning areas of interest. As the sky is not considered an area of interest, the real-time vertical FOV is slightly reduced. We also evaluate the end-to-end latency, which is the latency from point cloud generation to α LiDAR delivering the corrected point cloud to applications. The middle figure shows that α LiDAR has an average end-to-end latency about 35 ms, with a peak latency of around 60 ms. This performance confirms its suitability for real-time applications. In hybrid scenarios, latency increases due to a greater number of planar feature points among tall buildings, which elevates compute overhead due to the increased feature points requiring uncertainty calculation and updates.

7.3 Accuracy of Pose Estimation

We evaluate the performance of our state estimation method, the key module in α LiDAR. We compare it with two LiDAR-inertial odometry (LIO) algorithms: LIO-SAM [38], a conventional and widely used LIO algorithm; and Point-LIO [12], a recently proposed state-of-the-art LIO algorithm. We replace the state estimation module in α LiDAR with LIO-SAM and Point-LIO respectively, and record their state estimation sequences for evaluation. We compare the performance of these methods using the two metrics, ATE and RTE, as defined in Sec. 7.1.1. Table 2 shows the results on datasets with ground truth pose trajectory.

α LiDAR achieves a state trajectory error of 1.8 cm in indoor environments, surpassing the other two baselines by a factor of 90 and 7.5. LIO-SAM relies solely on IMU-based preliminary motion estimates and point cloud registration for state estimation, exhibiting poor performance during rapid LiDAR rotation. PointLIO improves upon LIO-SAM by tightly fusing LiDAR point clouds with IMU data in real time, achieving an approximate 8 $\times$ enhancement over LIO-SAM. However, PointLIO treats the uncertainty uniformly across all points, leading to suboptimal performance in our system. In contrast, our state estimation method individually estimates and updates the uncertainty for each point, resulting in more precise uncertainty estimation and significantly improving accuracy and robustness in challenging conditions. Our method exhibits slightly higher errors in outdoor scenarios due to the open nature of the tested environments, such as expansive roads, where close-range features are sparse. Consequently, the accuracy of pose estimation algorithms is primarily influenced by the precision of LiDAR measurements at long distances. As a result, all three methods demonstrate similar levels of accuracy.

7.4 System Overhead

We measure the run-time latency of α LiDAR’s individual steps through a continuous real-world operation trace. we assess latency across different levels of compute devices, including a Raspberry Pi and an Intel NUC11 with Core i7-1165

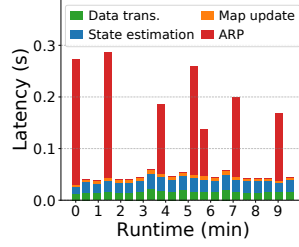


Figure 10: Run-time latency of a sequence for α LiDAR.

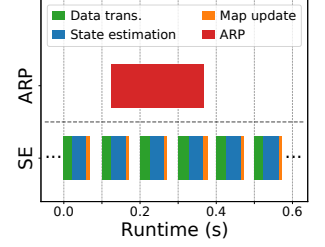


Figure 11: Detailed latency for α LiDAR’s two concurrent modules.

CPU. We evaluate the latency of two software modules of α LiDAR: state estimation (SE) and adaptive rotation planning (ARP). SE latency includes sensor data transmission, computation for LiDAR state estimation and point cloud correction, and local map updates. Fig. 10 illustrates the latency sequence over a 10-minute period. ARP is triggered when a new ROI is specified, and we randomly set different ROIs during this timeframe. A detailed timeline of the process for a newly specified ROI is presented in Fig. 11.

α LiDAR’s concurrent architecture ensures uninterrupted execution of state estimation, allowing for continuous output of corrected point clouds. On a Raspberry Pi, it achieves an average delay of less than 90 ms. Such efficiency is crucial for real-time applications such as object detection that rely on point cloud data. Adaptive rotation planning exhibits an average latency of approximately 0.28 s, enabling a swift shift in scanning focus to a new ROI, thereby achieving near real-time responsiveness.

For power consumption, α LiDAR only adds approximately 1.25 W–1.05 W for the motor and 0.2 W for the control board. This represents only a 12% increase compared to the typical 9.95 W consumed by conventional LiDAR sensors.

7.5 Benefits for SLAM

In this section, we evaluate the benefits α LiDAR can bring to a typical LiDAR application, simultaneous localization and mapping (SLAM). The key observations from this section are: (i) The rotation of α LiDAR significantly improves the precision, coverage, density, and feature richness of the constructed 3D map. (ii) Through adaptive rotation planning, α LiDAR achieves more uniform point distribution and reduces the occurrence of invalid points, thereby boosting mapping efficiency and uniformity, and further improving the metrics mentioned in (i).

We compare four LiDAR systems: (i) a Hesai Pandar-XT16 16-line mechanical LiDAR (16Line), (ii) a Livox MID360 solid-state LiDAR (Solid), (iii) α LiDAR with a Hesai Pandar-XT16 16-line LiDAR sensor, and (iv) α LiDAR without adaptive rotation planning (w/oARP), i.e., a 360-degree uniformly rotating Hesai Pandar-XT16 LiDAR system. We deploy our state estimation algorithm across all four systems as a SLAM

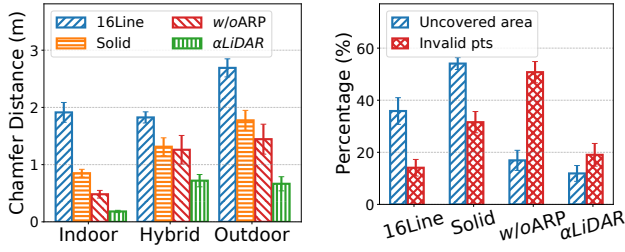


Figure 12: Mapping precision of different LiDAR systems under different environment settings.

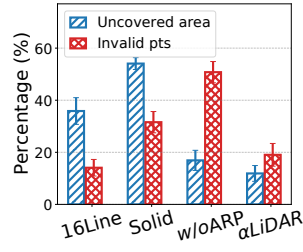


Figure 13: Map coverage and mapping efficiency of different LiDAR systems.

front-end. This is combined with the back-end map optimization method detailed in [20] to establish a complete SLAM framework. We employ each LiDAR system to conduct SLAM on the Lab, Corridor, Buildings, Garage and Road datasets, encompassing indoor, outdoor, and hybrid environmental settings. We use the metrics defined in Sec. 7.1.2 to evaluate their mapping performance.

According to Fig. 12, α LiDAR produces the most accurate 3D map, with an average Chamfer Distance of around 0.5 m. Conventional LiDAR systems (16Line and Solid) have FOV, resulting in fewer captured features for point cloud alignment and potential mapping errors. The baseline system without adaptive rotation planning (w/oARP) captures more features through rotation, but it is prone to capturing invalid points, such as those directed towards the sky in outdoor environments, which hinders further improvements in mapping accuracy. In contrast, α LiDAR’s adaptive rotation ensures that most points are directed towards valid areas, enabling enhanced feature capture and more accurate mapping.

Similarly, the limited FOV of conventional LiDARs restricts comprehensively mapping of environments, leaving substantial areas uncovered. However, this constrained FOV contributes to a high mapping efficiency (see Fig. 13), as most points are directed towards relevant regions. In contrast, the w/oARP system exhibits extensive map coverage owing to its rotational capabilities. Yet, its mapping efficiency decreases in outdoor environments due to the LiDAR sensor rotating towards the sky while the moving carrier (e.g., a robot or a vehicle) advances, resulting in areas being left unmapped. This highlights the importance of adaptive rotation strategies, which optimize the utility of each scanned point by adjusting rotation angles based on environmental context and carrier movement.

Fig. 14 shows that α LiDAR exhibits the highest density and a more consistent density distribution compared to the baseline systems. Fig. 15 further illustrates that all three baseline systems have suboptimal map uniformity, primarily due

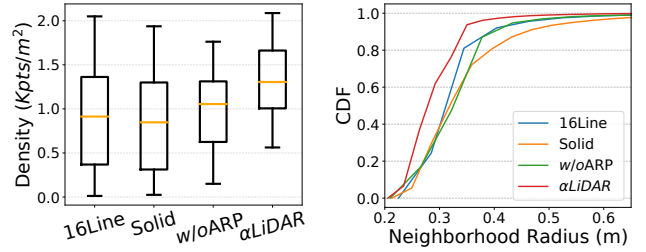


Figure 14: Map density of different LiDAR systems.

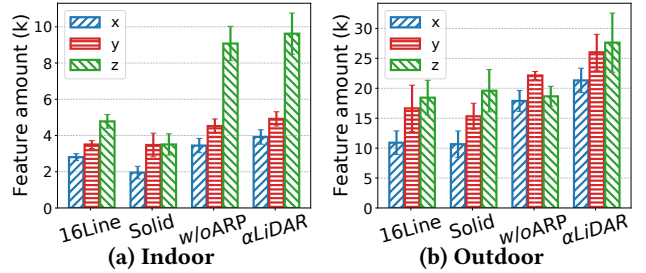


Figure 15: Uniformity of different LiDAR systems.

Figure 16: Spatial distribution of feature points detected by different LiDAR systems.

to ‘gaps’ within their 3D maps. These gaps can occur between scan lines in mechanical LiDARs or at the boundaries of point clouds in solid-state LiDARs. The w/oARP system may have gaps in areas not covered when the LiDAR rotates towards the sky, resulting in non-uniform 3D maps. In contrast, α LiDAR’s adaptive rotation effectively mitigates these gaps, leading to more consistent and uniform 3D mapping outcomes.

Additionally, we evaluate the quantity of feature points within each 3D map, which is an crucial metric for determining a map’s utility. Fig. 16 shows the distribution of features across three axes, x-axis (forward direction of the carrier), y-axis (right direction of the carrier), and z-axis (perpendicular to the ground). In indoor settings, both the w/oARP system and α LiDAR exhibit a notable increase in the number of features along the z-axis. This is due to the limited vertical FOV of 16-line and solid-state LiDARs, which restricts their ability to fully capture features along the z-axis. Conversely, both the w/oARP system and α LiDAR effectively scan ceilings and floors, which are rich in plane feature points. In outdoor settings, α LiDAR shows a consistent increase in feature points across all directions compared to other baselines.

7.6 Benefits for Obstacle Perception

In this section, we explore the improvements that α LiDAR brings to another typical LiDAR application, obstacle perception. The key findings are: (i) The adaptive rotation of α LiDAR significantly extends the LiDAR sensor’s effective

sensing distance, enabling perception of obstacles in previously inaccessible areas. (ii) Additionally, for objects within the sensor’s original perception range, α LiDAR’s adaptive rotation significantly improves the uniformity of the point cloud data, offering a more detailed and comprehensive representation of the objects.

We follow the three baselines described in Section 7.5. For our system, α LiDAR, we define the ROI for obstacle perception as the 3D space around the robot, with heights ranging from 0 m to 1.5 m. We evaluate the performance of each LiDAR system in obstacle perception on the Garage dataset. The metrics specified in Section 7.1.3 are used to assess the performance of these systems.

The numerical results are summarized in Table 4, providing average values for key metrics including ROI density, effective sensing range, number of bounding boxes within the effective sensing range, and points per bounding box. Fig. 17 visualizes the ROI point clouds (red boxes) and far obstacle point clouds (green boxes) for the evaluated methods.

Compared to other baselines, α LiDAR achieves the highest point cloud density within the ROI, owing to our adaptive rotation planning which focuses LiDAR scans primarily within the ROI. However, the advantage we offer is marginal since, in obstacle perception tasks, the point clouds from other baselines also predominantly fall within the ROI. Our approach does not increase the number of points that the LiDAR can detect. Nonetheless, we demonstrate a significant enhancement in effective sensing distance. Compared to traditional 16-line LiDARs, α LiDAR increases both the nearest and furthest effective sensing distances by 54% and 67% respectively. This improvement is attributed to our adaptive rotation, which optimizes the distribution of LiDAR scans through a reciprocating scanning motion, resulting in a more uniform distribution of point clouds within the ROI, thus enhancing the effective sensing distance. Fig. 17 visually supports this advantage, showing that point clouds from traditional LiDARs (both 16Line and Solid) concentrate near the LiDAR sensor, while α LiDAR’s point cloud distribution is more uniform. Additionally, for distant obstacles (38, m from the LiDAR sensor), traditional 16-line LiDARs tend to capture points along the sides of vehicles, whereas α LiDAR provides a more 3D and higher-coverage obstacle point cloud. This feature allows the robot to detect more obstacles, both close and distant, thereby benefiting downstream tasks such as behavior decision-making and motion planning.

The baseline *w/o*ARP exhibits the poorest ROI density. This is because its rotation disperses LiDAR points beyond the ROI, reducing the number of points within it. However, it still outperforms traditional LiDARs in terms of effective sensing distance due to its rotational scanning pattern, achieving a more uniform distribution of points.

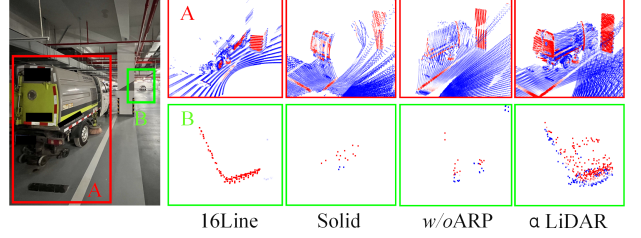


Figure 17: Comparative visualization of obstacle perception results from various LiDAR systems.

Table 4: Obstacle perception performance of different LiDAR systems.

Method	ROI density pts/m^3	ESD (m)		#bbx within ESD	#points in bbx
		Near	Far		
16Line	19.45	2.40	32.54	5.1	313.40
Solid	17.53	1.51	32.46	3.6	293.62
<i>w/o</i> ARP	16.79	1.59	42.03	5.5	275.87
α LiDAR	21.40	1.11	54.47	6.9	344.45

8 DISCUSSION AND CONCLUSION

Limitations of α LiDAR. α LiDAR relies on environmental features for state updates. In featureless environments, such as completely flat, open roads without structures or natural elements like buildings or trees, α LiDAR may encounter difficulties in tracking horizontal motion accurately. Additionally, α LiDAR’s performance degrades in adverse weather conditions such as rain, snow, or fog. Such limitation stems from the inherent optical characteristics of LiDAR technology, and is shared by almost all LiDAR systems.

Conclusion. In this paper, we present α LiDAR, a novel system that offers a panoramic field of view, high resolution, and dynamic focusing capabilities by integrating new mechanisms of controllable actuation, precise uncertainty-aware pose estimation, and adaptive rotation trajectory planning. The extensive real-world evaluations of α LiDAR prototypes underscore its superior performance in 3D mapping and sophisticated sensing tasks. In the future, we will develop new data compression and fusion algorithms that are specifically optimized for the panoramic 3D maps generated by α LiDAR.

Acknowledgments

We would like to thank our shepherd and the anonymous reviewers for their valuable feedback. This work was supported in part by National Key R&D Program of China under Grant No. 2023YFB4503700, Research Grants Council (RGC) of Hong Kong under General Research Fund under Grant 14222222, the Centre for Perceptual and Interactive Intelligence (CPII) under Grant EW01610 (RP4-3), NSFC under Grant No. 62372027, LY23F020026, 62303036 and Zhejiang Province Key R&D Program of China (2024C01071).

References

- [1] 2017. Velodyne LiDAR Launches VLS-128, The World's Highest Resolution LiDAR for Autonomous Vehicles. <https://velodynelidar.com/press-release/velodyne-lidar-launches-vls-128-the-worlds-highest-resolution-lidar-for-autonomous-vehicles/>.
- [2] Sick AG. 2006. LMS200/211/221/291 Laser Measurement Systems Technical Description. <https://sicktoolbox.sourceforge.net/docs/sick-lms-technical-description.pdf>.
- [3] Lucas R Agostinho, Nuno M Ricardo, Maria I Pereira, Antoine Hiolle, and Andry M Pinto. 2022. A practical survey on visual odometry for autonomous driving in challenging scenarios and conditions. *IEEE Access* 10 (2022), 72182–72205.
- [4] Mingcong Cao and Junmin Wang. 2020. Obstacle detection for autonomous driving vehicles with multi-lidar sensor fusion. *Journal of Dynamic Systems, Measurement, and Control* 142, 2 (2020), 021007.
- [5] Yi Cheng and Gong Ye Wang. 2018. Mobile robot navigation based on lidar. In *2018 Chinese control and decision conference (CCDC)*. IEEE, 1243–1246.
- [6] Cyberbotics. 2023. Webots: OPEN SOURCE ROBOT SIMULATOR. <https://cyberbotics.com>.
- [7] Renaud Dube, Andrei Cramariuc, Daniel Dugas, Hannes Sommer, Marcin Dymczyk, Juan Nieto, Roland Siegwart, and Cesar Cadena. 2020. SegMap: Segment-based mapping and localization using data-driven descriptors. *The International Journal of Robotics Research* 39, 2-3 (2020), 339–355.
- [8] Ryan Garnett and Matthew D Adams. 2018. LIDAR—A technology to assist with smart cities and climate change resilience: A case study in an urban metropolis. *ISPRS International Journal of Geo-Information* 7, 5 (2018), 161.
- [9] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 3354–3361. <https://doi.org/10.1109/CVPR.2012.6248074>
- [10] Pavel Gonzalez, Alicia Mora, Santiago Garrido, Ramon Barber, and Luis Moreno. 2022. Multi-lidar mapping for scene segmentation in indoor environments for mobile robots. *Sensors* 22, 10 (2022), 3690.
- [11] W Shane Grant, Randolph C Voorhies, and Laurent Itti. 2019. Efficient Velodyne SLAM with point and plane features. *Autonomous Robots* 43, 5 (2019), 1207–1224.
- [12] Dongjiao He, Wei Xu, Nan Chen, Fanze Kong, Chongjian Yuan, and Fu Zhang. 2023. Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry. *Advanced Intelligent Systems* (2023), 2200459.
- [13] Yuze He, Li Ma, Zhehao Jiang, Yi Tang, and Guoliang Xing. 2021. VI-eye: semantic-based 3D point cloud registration for infrastructure-assisted autonomous driving. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. 573–586.
- [14] Michael Helmberger, Kristian Morin, Beda Berner, Nitish Kumar, Giovanni Cioffi, and Davide Scaramuzza. 2022. The hilti slam challenge dataset. *IEEE Robotics and Automation Letters* 7, 3 (2022), 7518–7525.
- [15] HESAI. 2023. Pandar64: 64-Line Mechanical LiDAR. <https://www.hesaitech.com/cn/product/Pandar64>.
- [16] Hanjiang Hu, Xuxin Liu, Sharad Chitlangia, Akhil Agnihotri, and Ding Zhao. 2022. Investigating the Impact of Multi-LiDAR Placement on Object Detection for Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2550–2559.
- [17] Jianhao Jiao, Haoyang Ye, Yilong Zhu, and Ming Liu. 2021. Robust odometry and mapping for multi-lidar systems with online extrinsic calibration. *IEEE Transactions on Robotics* 38, 1 (2021), 351–371.
- [18] Nikhil Jonnavithula, Yecheng Lyu, and Ziming Zhang. 2021. Lidar odometry methodologies for autonomous driving: A survey. *arXiv preprint arXiv:2109.06120* (2021).
- [19] You Li and Javier Ibanez-Guzman. 2020. Lidar for autonomous driving: The principles, challenges, and trends for automotive lidar and perception systems. *IEEE Signal Processing Magazine* 37, 4 (2020), 50–61.
- [20] Xiyuan Liu, Zheng Liu, Fanze Kong, and Fu Zhang. 2023. Large-scale lidar consistent mapping using hierarchical lidar bundle adjustment. *IEEE Robotics and Automation Letters* 8, 3 (2023), 1523–1530.
- [21] Jordan Lorence. 2023. LIDAR Sensor Integration in Autonomous Vehicles Platforms. <https://www.mrlcg.com/resources/blog/lidar-sensor-integration-in-autonomous-vehicles-platforms/>.
- [22] LSLIDAR. 2023. C16: 16-Line Mechanical LiDAR. <https://www.leishen-lidar.com/cx/103>.
- [23] LSLIDAR. 2023. CH32: 32-Line Mechanical LiDAR. <https://www.leishen-lidar.com/ch/70>.
- [24] LSLIDAR. 2023. LS-S2. <https://www.leishen-lidar.com/tof/159>.
- [25] Bin Lv, Hao Xu, Jianqing Wu, Yuan Tian, Yongsheng Zhang, Yichen Zheng, Changwei Yuan, and Sheng Tian. 2019. LiDAR-enhanced connected infrastructures sensing and broadcasting high-resolution traffic information serving smart cities. *IEEE Access* 7 (2019), 79895–79907.
- [26] Flavio BP Malavazi, Remy Guyonneau, Jean-Baptiste Fasquel, Sebastien Lagrange, and Franck Mercier. 2018. LiDAR-only based navigation algorithm for an autonomous agricultural robot. *Computers and electronics in agriculture* 154 (2018), 71–79.
- [27] MARKETSSANDMARKETS. 2023. LiDAR Market. <https://www.market-sandmarkets.com/Market-Reports/lidar-market-1261.html>.
- [28] Sherif AS Mohamed, Mohammad-Hashem Haghbayan, Tomi Westerlund, Jukka Heikkonen, Hannu Tenhunen, and Juha Plosila. 2019. A survey on odometry for autonomous navigation systems. *IEEE access* 7 (2019), 97466–97486.
- [29] Matteo Palieri, Benjamin Morrell, Abhishek Thakur, Kamak Ebadi, Jeremy Nash, Arghya Chatterjee, Christoforos Kanellakis, Luca Carlone, Cataldo Guaragnella, and Ali-akbar Agha-Mohammadi. 2020. Locus: A multi-sensor lidar-centric solution for high-precision odometry and 3d mapping in real-time. *IEEE Robotics and Automation Letters* 6, 2 (2020), 421–428.
- [30] David Prokhorov, Dmitry Zhukov, Olga Barinova, Konushin Anton, and Anna Vorontsova. 2019. Measuring robustness of Visual SLAM. In *2019 16th International Conference on Machine Vision Applications (MVA)*. IEEE, 1–6.
- [31] RoboSense. 2023. Helios Series: A new generation of customized LiDAR platform, including 32-beam and 16-beam LiDAR. <https://www.robosense.ai/en/rslidar/RS-Helios>.
- [32] RoboSense. 2023. M1 & M1 Plus: Delivering automotive-grade lidar for smarter, safer vehicles. <https://www.robosense.ai/en/rslidar/rs-lidar-M1>.
- [33] RoboSense. 2023. Ruby Plus: Upgraded 128-beam LiDAR customized for L4 autonomous vehicle commercial operations. https://www.robosense.ai/en/rslidar/RS-Ruby_Plus.
- [34] Santiago Royo and Maria Ballesta-Garcia. 2019. An overview of lidar imaging systems for autonomous vehicles. *Applied sciences* 9, 19 (2019), 4093.
- [35] Schmersal. 2003. Schmersal Newsletter 02 Automotive. <https://docplayer.org/16519398-Schmersal-newsletter-02-automotive.html>.
- [36] Tixiao Shan and Brendan Englot. 2018. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 4758–4765.

- [37] Tixiao Shan and Brendan Englot. 2018. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 4758–4765.
- [38] Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus. 2020. Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 5135–5142.
- [39] TechCrunch. 2017. Toyota Research Institute debuts its next-generation automated driving platform. <https://techcrunch.com/2017/09/27/toyota-research-institute-self-driving-2-1/>.
- [40] Los Angeles Times. 2015. Ford driverless car to hit california roads. <https://www.latimes.com/business/autos/la-fi-hy-ford-driverless-car-to-hit-california-roads-20151215-story.html>.
- [41] u-blox. 2023. ZED-F9P module. <https://www.u-blox.com/en/product/zed-f9p-module>.
- [42] Inc. Velodyne LiDAR. 2015. HDL-64E User Manual. http://www.cajunbot.com/wiki/images/8/86/HDL-64E_Users_Manual_lowres_B.pdf.
- [43] Inc. Velodyne LiDAR. 2019. VLP-16 User Manual. <https://velodynelidar.com/wp-content/uploads/2019/12/63-9243-Rev-E-VLP-16-User-Manual.pdf>.
- [44] VentureBeat. 2021. Pony.ai raises \$100 million more to advance its autonomous vehicle tech. <https://venturebeat.com/business/pony-ai-raises-100-million-more-to-advance-its-autonomous-vehicle-tech/>.
- [45] Han Wang, Chen Wang, Chun-Lin Chen, and Lihua Xie. 2021. F-loam: Fast lidar odometry and mapping. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 4390–4396.
- [46] Weimin Wang, Ken Sakurada, and Nobuo Kawaguchi. 2017. Reflectance intensity assisted automatic and accurate extrinsic calibration of 3d lidar and panoramic camera using a printed chessboard. *Remote Sensing* 9, 8 (2017), 851.
- [47] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. 2022. Fast-lío2: Fast direct lidar-inertial odometry. *IEEE Transactions on Robotics* 38, 4 (2022), 2053–2073.
- [48] Wei Xu and Fu Zhang. 2021. Fast-lío: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter. *IEEE Robotics and Automation Letters* 6, 2 (2021), 3317–3324.
- [49] Tao Yang, You Li, Cheng Zhao, Dexin Yao, Guanyin Chen, Li Sun, Tomas Krajník, and Zhi Yan. 2022. 3D ToF LiDAR in mobile robotics: A review. *arXiv preprint arXiv:2202.11025* (2022).
- [50] Haoyang Ye, Yuying Chen, and Ming Liu. 2019. Tightly coupled 3d lidar inertial odometry and mapping. In *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 3144–3150.
- [51] Chongjian Yuan, Xiyuan Liu, Xiaoping Hong, and Fu Zhang. 2021. Pixel-level extrinsic self calibration of high resolution lidar and camera in targetless environments. *IEEE Robotics and Automation Letters* 6, 4 (2021), 7517–7524.
- [52] Zikang Yuan, Fengtian Lang, Tianle Xu, and Xin Yang. 2022. Sr-lío: Lidar-inertial odometry with sweep reconstruction. *arXiv preprint arXiv:2210.10424* (2022).
- [53] Ji Zhang and Sanjiv Singh. 2014. LOAM: Lidar odometry and mapping in real-time.. In *Robotics: Science and systems*, Vol. 2. Berkeley, CA, 1–9.
- [54] Ji Zhang, Sanjiv Singh, et al. 2014. LOAM: Lidar odometry and mapping in real-time.. In *Robotics: Science and systems*, Vol. 2. Berkeley, CA, 1–9.
- [55] Weikun Zhen, Yaoyu Hu, Jingfeng Liu, and Sebastian Scherer. 2019. A joint optimization approach of lidar-camera fusion for accurate dense 3-d reconstructions. *IEEE Robotics and Automation Letters* 4, 4 (2019), 3585–3592.
- [56] Pengwei Zhou, Xuexun Guo, Xiaofei Pei, and Ci Chen. 2021. T-loam: truncated least squares lidar-only odometry and mapping in real time. *IEEE Transactions on Geoscience and Remote Sensing* 60 (2021), 1–13.

A Artifact Appendix

A.1 Abstract

Our artifact provides a full-stack implementation of the controllable rotating LiDAR system, α LiDAR. α LiDAR enhances existing LiDAR sensors by incorporating the rotation of the LiDAR sensor in the direction where the LiDAR sensor’s field of view (FOV) is inherently limited, achieving panoramic FOV and high resolution. A significant challenge with α LiDAR is that rapid rotations result in highly distorted point clouds. Our solution focuses on accurately estimating the LiDAR sensor’s pose during rapid rotations to effectively correct and reconstruct the point cloud.

This artifact includes 1) Hardware Design: We have developed a hardware platform that supports controllable rotation, which includes a LiDAR sensor, a motor, an IMU, a stand, and a control board. We open-source all our materials for building this hardware platform from scratch, including the PCB design (PCB Gerber files) of the control board, the custom firmware of the MCU on the control board, CAD files of the mechanical parts, GCodes for 3D printing the mechanical parts, and sensor models. Moreover, we provide documents that detail the assembly of the hardware setup to facilitate user-friendly hardware reproduction. 2) Software Solution: We offer C++ and Python code that implements a pose estimation algorithm. This software runs on the carriers of α LiDAR, such as robots or vehicles, and corrects distorted point clouds by accurately estimating the LiDAR sensor’s pose. The software package also includes tools for evaluating and visualizing the performance of the pose estimation algorithm. 3) Dataset Provision: Recognizing the potential complexity of assembling the hardware, we also provide a dataset to facilitate the software evaluation. This dataset includes raw LiDAR point clouds, IMU measurements and motor encoder positions, which serve as inputs for the pose estimation algorithm. For more detailed documentation, please refer to our GitHub repository: https://github.com/HViktorTsoi/alpha_lidar

A.2 Contribution

We provide the implementation of both hardware and software for α LiDAR, the first LiDAR system that employs controllable rotation to achieve panoramic FOV and high resolution. The contributions of the artifact can be summarized as follows:

- We public a comprehensive tutorial on hardware buildup for active LiDAR systems. Particularly, the circuit design and firmware development for the control board, are carefully engineered to achieve precise time synchronization across multiple heterogeneous sensors and to coordinate

multi-sensor data transmission. Notably, this resource is the first and only public guide that facilitates simultaneous synchronization and coordination of motor, LiDAR, camera, and IMU, using an cheap, single-threaded MCU. While designed for the α LiDAR project, its applicability extends to other research involving active sensing or systems that require a compact, integrated solution for managing multiple sensors.

- Additionally, we release the code for our pose estimation algorithm designed to handle the rapid rotation of LiDAR sensors. We note that the rapid rotation of α LiDAR is far more intense than the motions typically encountered in standard LiDAR applications, such as SLAM. Importantly, our algorithm is not confined to handling rotations alone but is also capable of generalizing to other severe motions of LiDAR sensors. By sharing this code, we aim to foster further advancements in the critical research problem of pose estimation, thereby benefiting not only our community but also extending its impact to broader fields such as Robotics.

A.3 Artifact check-list (meta-information)

- **Algorithm:** An algorithm that estimates LiDAR's pose under rapid rotation. The algorithm takes raw LiDAR point clouds, IMU measurements and motor encoder positions as inputs, and outputs the pose estimation of LiDAR and the un-distorted point cloud.
- **Compilation:** GCC/G++ 7.5.0 to compile the C++ code. The compiler comes pre-installed with the Ubuntu 18.04 system.
- **Data set:** A sample dataset to evaluate the software part of α LiDAR. It is uploaded to Google Drive and the approximate size is 5GB.
- **Run-time environment:** Ubuntu 18.04 with ROS Melodic, GCC 7.5.0 and Python 3.8 installed.
- **Hardware:** PC or laptop with 4+ CPU cores and a NVIDIA GPU (for result visualization only).
- **Metrics:** Latency, trajectory error and FoV gain.
- **Output:** Qualitative results: Estimated LiDAR poses and un-distorted LiDAR point clouds. Quantitative results: Latency and trajectory error of the state estimation algorithm, and FoV gain of the LiDAR system.
- **Experiments:** README and scripts in the Github repository.
- **How much disk space required (approximately)?:** Around 12 GB.
- **How much time is needed to prepare workflow (approximately)?:** 2 hours.
- **How much time is needed to complete experiments (approximately)?:** 0.5 hour.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** MIT License.
- **Data licenses (if publicly available)?:** MIT License.
- **Archived (provide DOI)?:** DOI: 10.5281/zenodo.13757421

A.4 Description

A.4.1 How to access.

Clone a repository from https://github.com/HViktorTsoi/alpha_lidar. The approximate disk space usage is 12GB.

A.4.2 Hardware dependencies.

- PC or laptop with 4+ CPU cores.
- A NVIDIA GPU (for result visualization only).

A.4.3 Software dependencies.

- OS: Ubuntu 18.04.
- Compiler: GCC 7.5.0.
- ROS melodic distribution.
- Python 3.8.5.

A.4.4 Data sets.

The dataset (about 5GB) is collected by ourselves from our proposed active controllable rotating LiDAR system. You can access the dataset through the download link available at https://github.com/HViktorTsoi/alpha_lidar.

A.5 Installation

- Prepare the directory structure by
`mkdir -p alpha_lidar_ws/src && cd alpha_lidar_ws/src`
- Clone the repository
`git clone https://github.com/HViktorTsoi/alpha_lidar`
- Compile the C++ code by
`cd .. && catkin_make \`
`-DCATKIN_WHITELIST_PACKAGES="alpha_lidar"`
- Set up the environment by
`source ./devel/setup.zsh`

A.6 Experiment workflow

- In one terminal, launch the state estimation algorithm by
`roslaunch alpha_lidar state_estimation.launch`
then a GUI window should appear.
- In another terminal, replay the dataset and activate the state estimation by
`rosbag play /path/to/downloaded/dataset.bag`

During the experiment, the GUI window will display the trajectory of the LiDAR pose estimated in real time, as well as a constructed 3D map by integrating the un-distorted point clouds. Numerical results will be printed in the terminal.

A.7 Evaluation and expected results

When running the state estimation codes in the workflow, you will obtain the following outputs:

- Visualization of the trajectory of the LiDAR pose estimated in real time and reconstructed LiDAR point cloud. The performance of the pose estimation algorithm can be qualitatively verified by inspecting the quality (completeness, density, precision) of the point cloud.

- Numerical results: The latency of the pose estimation algorithm, the estimation error of the pose trajectory, and the benefits that α LiDAR brings to LiDAR sensors, such as the expansion of the field of view, etc.

A.8 Notes

We will continuously maintain the code and documentation in the GitHub repository <https://github.com/HViktorTsoi/a>

lpha_lidar. For more detailed steps and issues encountered during the evaluation process, please refer to this repository.

A.9 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>